

Collectors & Its Roles and Capabilities

Concept: Collectors in Java 8 are utility methods used in conjunction with `Stream.collect()` to accumulate elements into collections, concatenate strings, compute statistics, and more. Collectors offer a rich set of ready-made reductions to handle various data aggregation needs.

Roles:

1. **Transform Data** – Change elements from one form to another.
2. **Aggregate Data** – Summarize or count elements.
3. **Group Data** – Classify elements into groups.
4. **Partition Data** – Split data into two categories based on a predicate.

Capabilities Examples:

- **toList(), toSet(), toMap()**
- **joining()** – Concatenate strings.
- **groupingBy()** – Group elements by a classifier.
- **partitioningBy()** – Split elements into two lists.
- **summarizingInt()** – Generate statistical summary.

Example:

```
Map<Character, List<String>> grouped = Arrays.asList("apple", "banana",  
"avocado")  
    .stream()  
    .collect(Collectors.groupingBy(s -> s.charAt(0)));  
System.out.println(grouped);
```

Here are some practical scenarios:

1. Converting Streams to Collections:

```
List<String> fruits = Stream.of("apple", "banana", "mango")  
    .collect(Collectors.toList());  
Set<String> uniqueFruits = Stream.of("apple", "banana", "apple")  
    .collect(Collectors.toSet());
```

2. Mapping to a Map:

```
Map<String, Integer> nameLengthMap = Stream.of("Alice", "Bob", "Charlie")
    .collect(Collectors.toMap(name -> name, String::length));
```

3. Joining Strings:

```
String joined = Stream.of("A", "B", "C")
    .collect(Collectors.joining(", ", "[", "]")); // [A, B, C]
```

4. Grouping Data:

```
Map<Integer, List<String>> byLength = Stream.of("one", "two", "three",
    "four")
    .collect(Collectors.groupingBy(String::length));
```

5. Partitioning Data:

```
Map<Boolean, List<Integer>> partitioned = IntStream.rangeClosed(1, 10)
    .boxed()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
```

6. Summarizing Statistics:

```
IntSummaryStatistics stats = Stream.of(1, 2, 3, 4, 5)
    .collect(Collectors.summarizingInt(Integer::intValue));
System.out.println(stats.getAverage()); // 3.0
```